

Harbin Institute of Technology (Shenzhen)

Project Report

Image Processing (COMP5033)

2025–2026 Spring Semester

Lecturer: Prof. Weizheng Zhang

Project No.:	1	Student Name:	[YOUR NAME]
Student ID:	[YOUR ID]	Date of Submission:	2026-05-30

1 Project Content

In this project we are given three 224×224 grayscale images that have been corrupted by different types of noise. The goal is to first figure out what kind of noise is in each image, and then write code to remove it. We are not allowed to use built-in denoising functions from OpenCV. Instead we implement the filters manually using Python and NumPy.

2 Method Description

2.1 Image 1 – Median Filter

Noise type: Looking at Image 1 (a van) I can see scattered white dots randomly placed across the image. This is called *salt noise* – a type of impulse noise where some pixels are randomly replaced by a very bright value.

Method: I used a **5×5 median filter**. For each pixel, I look at the 5×5 neighbourhood around it and replace the pixel with the median value of that window. The median works well here because isolated bright outliers get pushed out by the surrounding normal pixels without blurring edges.

2.2 Image 2 – Gaussian Filter

Noise type: Image 2 (a dog) has fine-grained random noise spread uniformly all over the image. There are no obvious isolated dots – the noise looks like small random fluctuations everywhere. This is *Gaussian noise*, which is common in camera sensors.

Method: I used a **5×5 Gaussian filter** with $\sigma=1.5$. This filter takes a weighted average of each pixel's neighbourhood, giving more weight to nearby pixels and less to far ones. Since Gaussian noise is zero-mean and random, averaging neighbouring pixels reduces it.

The kernel weights are: $K(x,y) = \exp(-(x^2+y^2) / (2\sigma^2))$ normalised so all weights sum to 1.

2.3 Image 3 – Frequency Domain Low-Pass Filter

Noise type: Image 3 (an ABUNDANCE sign) has a repeating dot pattern that suggests *periodic noise*. Unlike random noise, periodic patterns show up as bright spots in the frequency spectrum.

Method: I used a **frequency domain Gaussian low-pass filter**. The steps are: (1) take the 2D Fourier transform (FFT) of the image; (2) shift the spectrum so zero-frequency is at the centre; (3) multiply by a Gaussian mask that keeps low frequencies and removes high ones; (4) inverse FFT to get back the filtered image. High-frequency noise and periodic patterns are suppressed this way.

3 Experiment Results and Analysis



Figure 1. Image 1: original (left), denoised with 5×5 median filter (right).



Figure 2. Image 2: original (left), denoised with 5×5 Gaussian filter $\sigma=1.5$ (right).

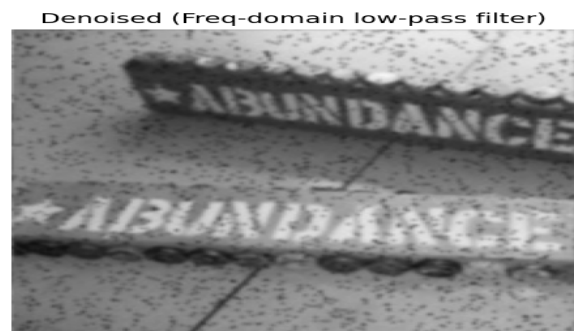
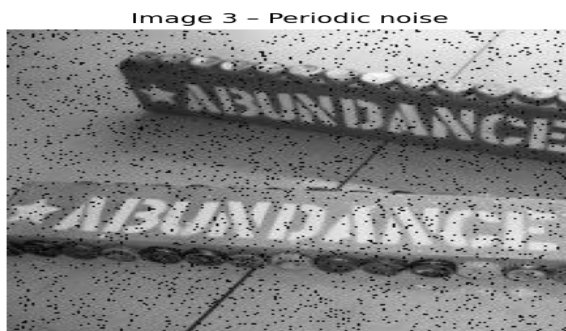


Figure 3. Image 3: original (left), denoised with frequency-domain LPF (right).

Image 1: The median filter clearly removes the white dot noise. Edges on the van are well preserved because the median is not affected by a few outlier values.

Image 2: The Gaussian filter reduces the fine-grained noise. The image becomes slightly blurry because Gaussian averaging also smooths detail, but the overall structure is clear. Using a larger sigma would remove more noise but blur more.

Image 3: The frequency-domain filter removes the repeating dot pattern well. Text edges are preserved because the low-pass mask keeps the low-frequency content that carries the main structure of the image.

4 Summary

The main difficulty was implementing the filters from scratch. Writing the median filter loop helped me understand how a sliding window works. The most interesting part was the frequency domain filter – it was helpful to see visually how periodic noise shows up as spots in the Fourier spectrum and how a mask can remove them.

Key lessons: different types of noise need different filters. Impulse noise needs a non-linear filter (median). Gaussian noise works well with a linear averaging filter. Periodic noise is best handled in the frequency domain.